PYTHON PROGRAMMING ASSIGNMENT 3

Implementation of Python Data Structures

Student Detail	Information
Name	Niteesh R.
Register Number	1924260341
Department	CSE AI & DS
Python Version	Python 3.12 (target environment)
Nature	Individual
Total Marks	50
Submission date in assignment	03/11/2026
GitHub link	Paste your actual repository URL here

Simple student-level English | Questions Q1–Q7 | Generated 05 November 2026

Declaration and Submission Note

This report presents the design, implementation, analysis and evaluation of seven Python data-structure tasks. The accompanying ZIP contains seven separate Python programs, a README, requirements file and saved text outputs.

Important: The GitHub URL is a placeholder because only you can create and share a repository from your GitHub account. After creating it, paste the real URL on this cover page and in Google Classroom. Run each program on your own computer, capture genuine screenshots and add them to the task folders.

Repository and Execution

Repository name: **CSA08-1924260341-Assignment3**. Python programs use the standard library only. Run from the repository folder:

```
python Task01/q1_rainfall.py
python Task02/q2_tuples_copying.py
python Task03/q3_dictionary_complaints.py
python Task04/q4_sets_electives.py
python Task05/q5_comprehensions_sensors.py
python Task06/q6_railway_timetable.py
python Task07/q7_contact_merge.py
```

Q1 – Lists and Slicing: Daily Rainfall Records (6 Marks)

Design and implementation: A list stores 365 daily rainfall readings in date order. The program uses slices to calculate weekly totals, finds the maximum and minimum weekly totals with their week positions, calculates a 7-day moving average, and lists zero-rainfall days.

```
weekly = [sum(readings[i:i+7]) for i in range(0, len(readings), 7)]
wettest = weekly.index(max(weekly))
driest = weekly.index(min(weekly))
dry_days = [i+1 for i, x in enumerate(readings) if x == 0]
```

Moving-average pseudocode

```
IF length(readings) < 7: return []
total = sum(first 7 readings)
append total / 7 to result
FOR each new reading:
    total = total + new reading - oldest reading
    append total / 7
RETURN result
```

Slicing and list costs

• readings[7:14] returns seven values at indexes 7 to 13.

• readings[-7:] returns the final seven values.

• readings[::7] returns every seventh item beginning at index 0; it does not calculate weekly totals.

• append() is O(1) amortised; insert() is O(n); pop() at the end is O(1), while pop(i) is O(n); sort() is O(n log n) typical.

For n readings, the naive method performs six additions for each full seven-value window: 6(n−6) additions. The running-sum method uses six initial additions and then one addition plus one subtraction for each new day: 6 + 2(n−7) = 2n−8 updates. Both are O(n) for a fixed window, but running sum avoids repeated work.

Edge cases: Empty input and lists shorter than seven return an empty average list. **Late reading:** list.insert() costs O(n) because later items shift. For a 365-day list, a list is still simple and appropriate. A deque(maxlen=7) is suitable for a live stream; maintain a running sum for O(1) average updates. A date-keyed dictionary gives convenient date lookup but needs date ordering for weekly calculations.

Alternative complexity: Full list storage is O(n). A seven-item deque window is O(7) extra space. A dictionary storing all date readings is O(n). A naive moving average is O(7n), simplified to O(n); running sum is O(n).

Q2 – Tuples, Mutability and Copying (7 Marks)

Design: Represent each bus stop as a (latitude, longitude) tuple and use it as a dictionary key. Tuples are immutable and hashable if all elements are hashable. Lists are mutable and unhashable, so using a list as a dictionary key raises TypeError.

```
good = [[0 for _ in range(cols)] for _ in range(rows)]
# Each loop iteration creates a separate inner list.
```

Aliasing memory diagram

```
Incorrect: [[0] * 3] * 3
row 0 ■■
row 1 ■■■■> SAME inner list object
```

```
row 2 ■■

Correct: [[0 for _ in range(3)] for _ in range(3)]
row 0 ■■■> inner list A
row 1 ■■■> inner list B
row 2 ■■■> inner list C
```

The completed TODO uses the list comprehension shown above. The expected result is: bad grid changes in every row; good grid changes only in row zero; identity checks print True and False.

Copying and experiments

• Aliasing (b = a) creates a second name for the same list.

• Shallow copy (copy.copy(a) or a[:]) creates a new outer list but shares nested objects.

• Deep copy (copy.deepcopy(a)) copies nested objects too.

• A shallow copy is enough for flat lists or intentionally shared nested values. Use deep copy when nested lists must be independent.

• A function changes the caller's list because both parameter and caller refer to the same mutable object.

A tuple containing a list is not deeply immutable: the tuple cannot replace its list element, but the list itself can still be changed. The program demonstrates this, compares sys.getsizeof() for one-million-item list and tuple containers, and times creation and indexing. getsizeof() measures the container, not all referenced objects.

Alternative sparse grid: Use a dictionary keyed by (row, col), storing only non-empty zones. Average access is O(1), as with list indexing. A full list grid uses O(rows × cols) space; a sparse dictionary uses O(number of occupied zones), but has higher per-entry overhead.

Q3 – Dictionaries: Farmer Helpline Complaints (7 Marks)

Design: Convert text to lowercase words, remove punctuation, ignore stop words using a set and count words in a dictionary. Group complaints with a dictionary of lists.

```
counts[word] = counts.get(word, 0) + 1

def top(counts, k):
    return sorted(counts.items(), key=lambda item: (-item[1], item[0]))[:k]
```

The completed TODOs produce: [('seeds', 3), ('price', 2), ('received', 2)]. Price comes before received because both counts are 2 and price is alphabetically earlier.

Analysis

• A dictionary hashes a key to locate it in a hash table. Lookup, insertion and deletion are O(1) average, although worst case can be O(n).

• Dictionary counting takes O(n) average for n tokens. list.count() for each of d distinct words takes O(n × d).

• Python 3.7 and later preserve dictionary insertion order.

• keys(), values() and items() return dynamic views of keys, values and key-value pairs.

Use setdefault(district, []).append(text) to group complaints by district. The same pattern can group complaints by crop. The script tests empty text, uppercase/lowercase and punctuation, and compares dictionary counting, collections.Counter and list.count() on 1,000, 10,000 and 100,000 complaints. For 100,000 complaints, dictionary or Counter is the practical choice; list.count() repeatedly scans the token list.

Alternative complexity: Dictionary loop and Counter are O(n) average time and O(d) count storage. Repeated list.count() is O(n × d), with O(d) output storage.

Q4 – Sets: Elective Registration Clashes (7 Marks)

Design: Store register numbers in sets, which automatically remove duplicate registrations. Use intersection, symmetric difference, union and difference.

```
both = ai_elective & iot_elective
exactly_one = ai_elective ^ iot_elective
neither = all_students - (ai_elective | iot_elective)
```

The expected clash set is 192311004 and 192311009. Exactly-one students are 192311001, 192311002, 192311007 and 192311010. Neither students are 192311003, 192311005, 192311006 and 192311008.

Venn diagram interpretation and analysis

• Intersection (&): overlap of the two circles – students in both electives.

• Symmetric difference (^): non-overlapping parts – students in exactly one elective.

• Union (|): both circles together – students in at least one elective.

• Universe minus union: outside both circles – students in neither elective.

• Set membership is O(1) average; list membership is O(n). Sets cannot contain lists because lists are mutable/unhashable. Tuples can be set elements if their contents are hashable.

The program tests empty and identical electives, measures membership for 10,000, 100,000 and 1,000,000 register numbers, and compares nested loops with set intersection.

Alternative complexity: Nested loops cost O(n × m). Sorting both lists and walking through them costs O(n log n + m log m). Set operations take O(n + m) average time after conversion, with O(n + m) extra storage. Converting a list to a set loses duplicate counts and original order.

Q5 – List Comprehension: Sensor Reading Cleaning (6 Marks)

Design: Valid sensor values are between –50 °C and 50 °C inclusive. Convert valid values with F = C × 9/5 + 32. Alarm minutes are valid sensor readings outside the safe 2–8 °C range.

```
clean_c = [c for c in readings if -50 <= c <= 50]
clean_f = [c * 9/5 + 32 for c in readings if -50 <= c <= 50]
alarms = [minute for minute, c in enumerate(readings, 1)
          if -50 <= c <= 50 and not (2 <= c <= 8)]
```

General form: [expression for item in iterable if condition]. The program implements a loop with append, a list comprehension and map()/filter() with lambda-style functions, then verifies that all three return equal results. It tests all-valid and all-invalid input.

A comprehension is harder to read when it contains many nested loops or complicated conditions; use a normal loop then. Dictionary comprehension example: {i: c for i, c in enumerate(readings, 1) if -50 <= c <= 50}. Set comprehension example: {c for c in readings if -50 <= c <= 50}.

A list comprehension stores all results and uses O(n) memory. A generator expression produces one value at a time and generally uses O(1) extra memory. Both need O(n) time to process n readings. sys.getsizeof() shows the generator object is much smaller, but does not include future generated values.

Evaluation: The program times loop, comprehension and map/filter for 10,000, 100,000 and 1,000,000 readings. A comprehension is concise but not always fastest; measured performance depends on the interpreter and workload.

Q6 – Nested Data: Railway Timetable (7 Marks)

Design: The outer dictionary maps train number to a dictionary containing the train name, ordered stop list and running-day set. Each stop is a (station, arrival, departure) tuple.

```
timetable["12601"]["stops"][2][2]  # third station departure
```

Dictionary: fast train-number lookup. List: preserves stop order. Tuple: groups fixed station fields. Set: efficient running-day membership. The program builds five trains, prints the timetable, finds trains for a station/day and sorts them by departure time using sorted().

Without an index, a station query scans all trains and their stops: O(T × S), where T is train count and S is average stops per train. A station index maps station name to (train number, departure time) entries. The indexed query examines K trains at that station and takes O(K) average for the stored departure data, plus sorting O(K log K). Building the index costs O(total stops) time and space.

The experiment creates 100, 1,000 and 10,000 trains and compares scan and indexed queries. It tests an unknown station and a train not running on the selected day. The index uses extra memory but helps when many queries are made.

Alternative: A flat list of (train, station, arrival, departure, days) tuples scanned per query costs O(total stops). A station index reduces a query to O(trains at that station), at the cost of O(total stops) additional index entries.

Q7 – Contact Merging from First Principles (10 Marks)

Stage 1 – Interpret: Input: contact records from two phones and a SIM. Output: merged contacts with one record per confidently matched person and sets of all phone numbers and e-mails. The primary matching rule is a shared normalised phone number. A name alone is not enough because different people can have the same name.

Measurable requirements: spaces, hyphens and +91 country prefix should normalise consistently; duplicate numbers/e-mails should appear once; missing fields should not crash the program; contacts with different numbers must not merge just because their names match.

Stage 2 – Design and pseudocode

• Phone normalisation: keep digits only; remove 91 from a 12-digit Indian number; remove leading 0 from an 11-digit number; accept 10-digit mobile values as matching keys.

• Name normalisation: lowercase and keep alphabetic words. The implementation does not merge by name alone.

• Dictionary maps normalised phone number to a contact group. Each group stores phone numbers and e-mails in sets.

```
FOR each contact:
    normalise phone numbers and e-mails
    find existing groups for known phone numbers
    IF a matching group exists: merge into it
    ELSE IF a valid phone exists: create a group
    ELSE: create a separate no-phone group
    add all phone numbers and e-mails to the group
    update phone-to-group dictionary
RETURN sorted merged contacts
```

Stage 3 – Implementation and measurements

The program tests duplicate contacts with different name spellings, +91 and plain phone formats, multiple numbers, missing phone, missing name and duplicate e-mails. It verifies a small Ravi example by hand and measures 1,000 and 100,000 contacts.

In the hand-check, +91 98400 12345 and 9840012345 normalise to 9840012345. The merged record contains two unique phone numbers and three unique e-mail addresses.

Stage 4 – Defend the design

Dictionary lookup is O(1) average per phone key, so processing n contacts is O(n) average when each contact has a small bounded number of phone numbers. Comparing every pair is $O(n^2)$. The dictionary and output groups use O(n) space. Different people with the same name but different phone numbers remain separate. A limitation is that contacts without phone numbers cannot be safely merged; recycled/shared numbers can also create false matches.

Alternative pairwise method: Compare every contact with every other contact after normalising phone numbers. There are $n(n–1)/2$ possible pairs: 499,500 for 1,000 contacts and 4,999,950,000 for 100,000 contacts. Pairwise time is $O(n^2)$; dictionary grouping is O(n) average and needs O(n) space.

Experimental Results – Sample Execution

Saved output files contain full measurements from the available execution environment (Python 3.13.5). The assignment requests Python 3.12; rerun on your own Python 3.12 laptop and use your own timing values/screenshots. Results vary with CPU, interpreter and background load.

Q1 – Moving average

```
n=    365: naive=0.000074s, running-sum=0.000029s
n=  36500: naive=0.008216s, running-sum=0.003385s
n= 365000: naive=0.079130s, running-sum=0.035861s
```

Q3 – Word counting

```
complaints=  1000: dict=0.002012s, Counter=0.000625s, list.count=0.001121s
complaints= 10000: dict=0.020397s, Counter=0.003420s, list.count=0.009192s
complaints=100000: dict=0.211279s, Counter=0.029366s, list.count=0.099041s
```

Q4 – Set membership

```
n=  10000: list membership=0.00006108s, set membership=0.00000051s
n= 100000: list membership=0.00070237s, set membership=0.00000078s
n=1000000: list membership=0.00623291s, set membership=0.00000146s
```

Q5 – Sensor processing

```
n=  10000: loop=0.001614s, comprehension=0.002092s, map/filter=0.001250s
n= 100000: loop=0.014304s, comprehension=0.021191s, map/filter=0.014184s
n=1000000: loop=0.150956s, comprehension=0.224745s, map/filter=0.141661s
```

Q6 – Railway query

```
trains=  100: index-build=0.000045s, scan-query=0.000029s, indexed-query=0.000179s, equal=True
trains= 1000: index-build=0.000744s, scan-query=0.000332s, indexed-query=0.000121s, equal=True
trains=10000: index-build=0.007081s, scan-query=0.002891s, indexed-query=0.002147s, equal=True
```

Q7 – Contact merging

```
contacts=  1000: dictionary merge=0.003163s, output groups=1000
contacts=100000: dictionary merge=0.988348s, output groups=100000
```

These experiments support the expected trends: running sums reduce repeated additions; set membership avoids scanning a list; dictionary/Counter counting scales better than repeated list.count() for many distinct words; and a station index can reduce

repeated station-search work. Timing differences are not universal.

Overall Conclusion

Lists are useful for ordered editable sequences; tuples represent fixed records; dictionaries support key-based lookup and counting; sets store unique values and support set algebra; comprehensions create filtered collections; and nested structures represent related real-world data. An index uses extra memory to make repeated searches faster.

Viva Preparation

• Rainfall uses a list because readings are ordered by day and slicing is useful.

• Tuples can be dictionary keys because they are immutable and hashable when their contents are hashable.

• The bad grid repeats one inner list reference; the list comprehension creates separate rows.

• Dictionaries count words efficiently using average O(1) updates.

• Sets remove duplicates and make intersection easy.

• Generators save memory by producing one value at a time.

• A railway index avoids scanning every train for every station query.

• Contacts are matched by normalised phone, not name alone, because names are not unique.

Final Submission Checklist

• Create GitHub repository: CSA08-1924260341-Assignment3.

• Upload Task01–Task07, README.md, requirements.txt, outputs and this report.

• Run every file in Python 3.12 and take your own screenshots in each task folder.

• Commit progressively, with at least one commit per task.

• Add the actual GitHub URL to the PDF cover and submit the PDF and repository link in Google Classroom.

Academic integrity: Read and understand the code before submission. The instructor may ask you to explain or modify it during viva.